

Organização arquitetural pragmática do caso de uso SignIn

1. Objetivo

No caso de uso de autenticação, é comum surgir a dúvida: **onde deve ficar o objeto que recebe a requisição HTTP?**

Nesta disciplina, adotaremos o seguinte mapeamento para as camadas do Clean Architecture:

Camada CA	Camada no Diagrama
Entities	Domain
Use Cases	Application
Interface Adapters	Infra
Frameworks & Drivers	o framework usado

Nesse arranjo, o objeto que atua como **controller** deve ficar na camada **Infra**, porque ele funciona como um **adaptador entre a interface HTTP e o caso de uso**. Já os elementos que dependem diretamente do framework web (Fastify, Express, etc) pertencem à camada de tecnologia usada. Neste exemplo, usaremos o `Fastify` como framework web.

Essa decisão é coerente com um princípio simples: **as regras de negócio não devem depender do framework web**. O controller apenas recebe a entrada, chama o caso de uso e devolve a saída. As regras de autenticação permanecem na Application e no Domain.

2. Ideia central

Ao implementar uma rota de login, existem quatro tipos de responsabilidade diferentes:

1. **Receber a requisição HTTP**
2. **Traduzir a requisição para um formato que o sistema entende**
3. **Executar a regra de negócio**
4. **Devolver uma resposta HTTP**

Quando essas responsabilidades ficam misturadas em um único objeto, o código tende a ficar rígido, difícil de testar e fortemente acoplado ao framework.

A abordagem pragmática procura evitar isso com a menor complexidade possível.

3. Distribuição das responsabilidades

Domain

A camada **Domain** contém os conceitos centrais do negócio, independentes de banco, HTTP ou framework.

Exemplos no sign-in:

- `Email`
- `Password`

Esses objetos representam conceitos do domínio e ajudam a evitar validações soltas e repetidas pelo sistema.

Application

A camada **Application** contém o caso de uso e as portas de que ele precisa para funcionar.

Exemplos:

- `SignInUseCase`
- `LoadUserByEmail`
- `PasswordCompare`
- `AccessTokenGenerator`
- `SignInInput`
- `SignInOutput`

O `SignInUseCase` deve conhecer apenas o necessário para executar a autenticação:

- carregar o usuário
- comparar a senha
- gerar o token

Ele **não deve saber** que a entrada veio de uma rota Fastify, nem que a saída será enviada como resposta HTTP. Controllers devem ser finos e casos de uso não devem depender de objetos de transporte como request/reply.

Infra

Na convenção adotada aqui, a camada **Infra** corresponde ao papel de **Interface Adapters**. É nela que colocamos os objetos que fazem a ponte entre o mundo externo e o caso de uso.

Na versão pragmática, os objetos mais importantes são:

- `SignInController`
- `HttpRequest`
- `HttpResponse`
- `UserRepository`
- `BCryptAdapter`
- `JwtAdapter`
- `PostgresAdapter`

Papel do `SignInController`

O `SignInController` deve:

- receber um `HttpRequest`
- extrair os dados relevantes
- montar o `SignInInput`
- chamar o `SignInUseCase`
- transformar o resultado em `HttpResponse`

Ele **não implementa regra de negócio**. Seu papel é apenas de coordenação.

Papel de `HttpRequest` e `HttpResponse`

Esses objetos representam contratos internos simples para entrada e saída HTTP. Eles existem para evitar que o controller dependa diretamente de `FastifyRequest` e `FastifyReply`.

Isso permite trocar o framework no futuro, ou testar o controller sem precisar subir um servidor.

Tecnologia usada

A camada de **tecnologia usada** contém tudo que depende diretamente de bibliotecas e frameworks concretos.

Exemplos:

- `Fastify`
- `FastifySignInRoute`
- `FastifyControllerAdapter`
- `PostgreSQL`
- `BCrypt`
- `JsonWebToken`

Papel do `FastifySignInRoute`

Esse objeto registra a rota HTTP, por exemplo:

- `POST /auth/sign-in`

Ele define como o Fastify receberá a chamada e qual adapter será acionado.

Papel do `FastifyControllerAdapter`

Esse adapter converte:

- `FastifyRequest` → `HttpRequest`
- `HttpResponse` → `FastifyReply`

Assim, o Fastify conversa com o controller sem que o controller dependa do Fastify.

4. Fluxo completo da requisição

Na solução pragmática, o fluxo do login fica assim:

```
Fastify → FastifySignInRoute → FastifyControllerAdapter → SignInController → SignInUseCase
```

Depois disso, o caso de uso conversa com suas dependências:

- `LoadUserByEmail`
- `PasswordCompare`
- `AccessTokenGenerator`

E essas dependências são implementadas pelos adapters concretos:

- `UserRepository` → `PostgresAdapter`
- `PasswordCompare` → `BCryptAdapter`
- `AccessTokenGenerator` → `JwtAdapter`

Esse fluxo é importante porque preserva a direção das dependências: **as camadas externas dependem das internas, nunca o contrário**.

5. Estrutura mínima recomendada

Para o caso de uso de sign-in, a estrutura mínima recomendada é a seguinte.

Domain

- `Email`
- `Password`

Application

- `SignInUseCase`
- `LoadUserByEmail`
- `PasswordCompare`
- `AccessTokenGenerator`
- `SignInInput`
- `SignInOutput`

Infra

- `SignInController`
- `HttpRequest`
- `HttpResponse`
- `UserRepository`
- `PostgresAdapter`
- `BCryptAdapter`
- `JwtAdapter`

Tecnologia usada

- `Fastify`
- `FastifySignInRoute`
- `FastifyControllerAdapter`
- `PostgreSQL`
- `BCrypt`
- `JsonWebToken`

Essa organização é suficiente para atender ao requisito de uma API REST com autenticação, mantendo o sistema claro, testável e consistente com a proposta arquitetural do projeto.

6. O que o controller deve fazer

O controller deve ser simples. Em termos práticos, ele deve:

- ler os dados da requisição
- montar o input do caso de uso
- chamar o caso de uso
- devolver a resposta apropriada

Exemplo de responsabilidades adequadas do controller:

- pegar `email` e `password` do corpo da requisição
- chamar `SignInUseCase.execute(...)`
- devolver `200` com token em caso de sucesso
- devolver erro adequado em caso de falha

7. O que o controller não deve fazer

O controller **não deve**:

- consultar banco diretamente
- usar bcrypt diretamente
- gerar JWT diretamente
- decidir regra de autenticação
- validar regras de domínio
- conter lógica de negócio

Se o controller começar a fazer essas tarefas, ele deixa de ser um adaptador e passa a concentrar responsabilidades demais.

8. Por que essa solução é chamada de pragmática

Ela é pragmática porque mantém apenas os elementos que realmente agregam valor arquitetural:

- separa framework de regra de negócio
- mantém o caso de uso independente
- evita acoplamento direto ao Fastify
- facilita testes
- não cria abstrações em excesso

Ou seja, não se trata de “usar Clean Architecture de forma máxima”, mas de usar apenas o que é necessário para preservar fronteiras importantes. Isso está alinhado à diretriz do projeto de evitar aplicação dogmática da arquitetura e abstrações sem valor claro.

9. Regra prática

Ao modelar um endpoint, use a seguinte regra:

- Se o objeto conhece **regra de negócio**, ele tende a estar em **Application** ou **Domain**
- Se o objeto conhece **HTTP**, mas não conhece Fastify diretamente, ele tende a estar em **Infra**
- Se o objeto conhece **Fastify**, **PostgreSQL**, **bcrypt** ou outra biblioteca concreta, ele pertence à camada de **tecnologia usada**

Essa regra resolve a maior parte das dúvidas de posicionamento.

10. Conclusão

Na convenção adotada nesta disciplina, o **controller do sign-in deve ficar em Infra**, porque ele é um adaptador entre a interface HTTP e o caso de uso.

Já os objetos que dependem diretamente do Fastify devem ficar em **tecnologia usada**.

A solução mínima recomendada para o endpoint de login é:

- uma rota do Fastify
- um adapter do Fastify para o controller
- um controller simples
- um caso de uso isolado
- adapters concretos para banco, hash e token

Esse desenho é suficiente para manter boa organização, clareza e testabilidade, sem tornar o projeto artificialmente complexo.

Glossário de termos técnicos

Adapter Objeto que faz a conversão entre dois formatos ou duas interfaces diferentes. Exemplo: converter uma requisição do Fastify para um formato interno do sistema.

API REST Estilo de interface para comunicação entre cliente e servidor por meio de recursos HTTP, como `GET`, `POST`, `PUT` e `DELETE`. O projeto adota backend exposto por API REST.

Application Camada onde ficam os casos de uso. Coordena o comportamento da aplicação sem depender de framework web ou banco de dados.

BCrypt Biblioteca usada para hash e comparação segura de senhas.

Controller Objeto que recebe a entrada da interface externa, chama o caso de uso e devolve a saída. Deve ser fino e sem regra de negócio.

Dependency direction Princípio segundo o qual as dependências devem apontar para dentro da arquitetura, isto é, das camadas mais externas para as mais internas.

Domain Camada que contém os conceitos centrais do negócio, como entidades e value objects. Deve permanecer independente de HTTP, banco e frameworks.

DTO (Data Transfer Object) Objeto simples usado para transportar dados entre partes do sistema, sem carregar regra de negócio.

Fastify Framework web usado para expor a API HTTP.

Framework & Drivers / tecnologia usada Camada mais externa, onde ficam bibliotecas, frameworks, banco de dados e detalhes concretos de execução.

HTTP Protocolo de comunicação usado entre clientes e servidores web.

Infra Na convenção adotada aqui, é a camada que cumpre o papel de **Interface Adapters**. Ela conecta o mundo externo aos casos de uso.

Interface Adapters Elementos que adaptam formatos externos para formatos internos e vice-versa.

JWT (JSON Web Token) Formato de token comumente usado para autenticação e autorização em APIs.

Porta (Port) Contrato usado pela Application para declarar do que precisa, sem conhecer a implementação concreta.

Presenter Objeto responsável por transformar uma saída interna em um formato adequado de resposta.

Request / Response Objetos que representam, respectivamente, a entrada e a saída de uma interação HTTP.

Use Case Objeto que representa uma intenção clara do sistema, como autenticar um usuário. Cada caso de uso deve ter responsabilidade única.

Value Object Objeto de domínio definido mais por seu valor do que por identidade. `Email` e `Password` são exemplos típicos nesse contexto.